Streaming data can provide significant benefits for organizations that need to process large volumes of data quickly and make real-time decisions based on that data. However, there are transformations that require a more complex type of streaming, specifically, **stateful streaming**. Stateful streaming refers to stream processing that consumes a continuous data stream and persists the state of past events. Persisting the state allows for the stream to "know" information from previous transactions. This is particularly helpful when calculating aggregates over a time window since the aggregate would need values from the previous window in the stream. To make this clearer, we provide an example of this in the *Streaming Transactions* project. There is also an informative video linked in the *Further reading* section that explains stateful streaming in detail.

> **Note**
>
> Streaming feature tables should be altered before you begin writing to them. Running an `ALTER TABLE` command causes the stream to quit. However, you can restart the stream if you must alter the table. Be prepared and plan ahead as much as possible!

The DFE client supports intelligent handling and lookups for time-based features, such as our timestamped streaming features. Let's go over this time-saving product feature next.

Employing point-in-time lookups for time series feature tables

Time series feature tables are any table in Unity Catalog with a `TIMESERIES` primary key. These tables are eligible for point-in-time lookups, which is a mechanism for looking up the correct feature values. Before `training_sets`, coming in *Chapter 6*, we often joined tables to connect training rows with their feature values. However, a fine-grained event timestamp is not ideal for joining. This led to rounding the timestamps to minutes, hours, or even days. Depending on the use case, this method may or may not work. For example, joining on `TransactionTimestamp` in *Figure 5.3* is not realistic in a standard join so one might create `TransactionMinute` or `TransactionHour`.

TransactionTimestamp	TransactionMinute	TransactionHour
2023-09-03 19:23:09.765676	2023-09-03 19:23:00	2023-09-03 19:00:00
2023-09-03 19:23:09.765821	2023-09-03 19:23:00	2023-09-03 19:00:00
2023-09-03 19:23:09.765899	2023-09-03 19:23:00	2023-09-03 19:00:00

Figure 5.3 – Example of timestamp rounding for easy joining